



VALT · OWN THE MUSIC YOU LOVE

Milestone 3 — Security Audit Report

Automated Scanning, Manual Assessment & On-Chain Review

Project: Valt — Superfan Experience Engine | **Project Catalyst:** Fund 11, Project #1100019

Milestone: 3 — Implementation & Prelaunch | **Scope:** valt-platform 2.0.3, valt-theme, wallet-connection layer

Prepared: 16 July 2026 | **Updated:** 3 September 2026 (autonomous-style assessment added) | **Assessed by:** Awen LLC engineering

1. Scope, Method & Criterion Mapping

This report is the evidence for the Milestone 3 Output *"Perform security audits and implement necessary measures to ensure safety and integrity"*, and for two Acceptance Criteria: *"Passes all vulnerability checks conducted through automated cybersecurity tools (NodeZero or similar)"* and *"Passes all manual security assessments documented by the team"*.

Criterion	Status	Where
Passes all vulnerability checks via automated tooling (NodeZero or similar)	Met	\$3 — semgrep (122 rules / 73 files) in CI on every push, plus an autonomous-style ("NodeZero-equivalent") assessment (§3.1): 0 Critical, 0 High; the 1 Medium and material Lows resolved this milestone
Passes all manual security assessments documented by the team	Met	\$4 on-chain review; \$5 controls review; \$6 findings — see dispositions

Scope — corrected from an earlier draft

An earlier draft of this report placed the wallet-connection layer (the third-party **CardanoPress** plugin) out of scope as a vendor dependency. **That was the wrong call, and this version corrects it.** Valt's core security promise — that access to gated content follows on-chain ownership — is enforced *through* that layer. Auditing Valt's own gate while excluding the component that establishes who owns what is auditing the lock and ignoring the door. The wallet-connection and asset-verification path is therefore treated as in scope for this milestone, and \$4 reports what that review found.

- **In scope.** First-party code — the `valt-platform` plugin and the `valt-theme` child theme — plus the **wallet authentication and on-chain asset-verification path** that token-gating depends on, including the relevant CardanoPress surface.
- **Out of scope.** The internals of NMKR, Blockfrost and Stripe as services (trusted external providers), and unrelated third-party plugins (Pods, Elementor). The hosting perimeter is noted where relevant but not independently tested.
- **Network.** Cardano pre-production testnet. No mainnet funds are at risk in this milestone.

Method

Three passes. An **automated** pass (§3) using semgrep with maintained PHP and security-audit rule packs, wired into CI. An **on-chain / wallet-layer** pass (§4) tracing how a wallet is authenticated and how ownership is proven and cached. A **first-party manual** pass (§5–§6) reviewing every authentication and authorisation boundary, every place user input

reaches output or SQL, and every place a credential is read or stored. Structural invariants confirmed by hand were encoded as tests (`tests/SecurityTest.php`) so they cannot silently regress.

2. Summary of Results

The automated scan returned **no true positives**. The on-chain review found **two critical weaknesses in the third-party wallet-authentication layer** — an authorisation bypass and an unauthenticated account-takeover — which Valt reproduced, mitigated first-party, and **responsibly disclosed**. **The maintainer has since released a fix (CardanoPress 1.36.1) crediting Valt, and both local and production are updated**. The first-party manual assessment identified further issues, of which the higher-severity ones are resolved in this milestone and the remainder are accepted with documented rationale and gated as mainnet prerequisites.

This is a candid report, not a clean bill of health. The most valuable outcome of taking the wallet layer seriously was finding a class of flaw that a first-party-only audit would have missed entirely. No credential has ever been committed to the repository or its history, and the authorisation model in Valt's own code is applied consistently — every AJAX handler nonce-checked, every REST route with an explicit permission callback, no unauthenticated AJAX surface.

Note on scope evolution. Findings §6.3 and §6.6 concern the Stripe card-payment path present in `valt-platform 2.0.2`. After this assessment, **Stripe fiat checkout was removed entirely in `valt-platform 2.0.3`** — Valt is web3-native (NMKR only, which carries its own fiat option). Those Stripe findings and the Stripe credential notes in §5 are retained as the historical record of the milestone's security work; the module is no longer in the shipped code.

Area	Result
Automated SAST (semgrep)	0 true positives (1 triaged false positive)
Autonomous-style assessment (NodeZero-equivalent, §3.1)	0 Critical · 0 High · 1 Medium · 5 Low · 3 Info — Medium + material Lows resolved this milestone
Wallet layer (third-party)	2 critical — disclosed → fixed upstream in <code>v1.36.1</code> , Valt credited (§4)
First-party — resolved this milestone	CSRF, stored XSS, unsettled-payment mint, gating cache staleness
First-party — accepted with rationale	Unbounded pagination, DB-stored minting keys, idempotency race, no rate limiting

3. Automated Vulnerability Scan

The criterion names *"NodeZero or similar"*. NodeZero is an autonomous pentest platform aimed at network and infrastructure attack paths. Valt at this milestone is an application on managed hosting on a testnet, so the meaningful risk surface is application code. We selected **semgrep** — a widely used static application security testing tool — as the "or similar" instrument: its rule packs target the vulnerability classes that matter here, its output is reproducible by a reviewer from the repository, and it runs unattended in CI.

```
$ semgrep scan --config=p/php --config=p/security-audit code/
Ran 122 rules on 73 files: 1 finding.
errors: 0
```

Rule	Location	Triage
php.lang.security.injection.echoed-request	includes/admin-nft-monitor.php:119	False positive. The value is cast to integer on the line immediately above (<code>\$n = (int) \$_GET['uploaded'];</code>) before being echoed, so no injection is reachable. Recorded and explained rather than suppressed.

Supporting automated checks: `php -l` across all working trees (0 failures); committed-secret pattern scan (0 matches); dangerous-construct scan — `eval`, `unserialize`, `extract`, `shell_exec` — (0 occurrences). All asserted by `tests/SecurityTest.php` and re-run in CI on every push.

3.1 Autonomous-style assessment (NodeZero-equivalent)

To complement static analysis with an **attacker's-perspective** pass — the reasoning model a tool like NodeZero applies — a structured autonomous-style assessment was run over the full `valt-platform` 2.0.3 and `valt-theme` code, plus the site's must-use plugins. It reasons about reachable attack *paths* (who, how, what they gain) across broken access control, injection, CSRF, authentication trust boundaries, secrets exposure, SSRF, sensitive-data exposure, rate-limit abuse, and file handling — and, importantly, records what was checked and found **clean**, not only what was flagged.

Verdict: PASS — strong. No unauthenticated or low-privilege path to admin, artist, or minting actions was found; and no SQL injection, XSS, SSRF, or secrets exposure exists in the reviewed code. Every dynamic query uses `$wpdb->prepare()`, output is systematically escaped, each custom AJAX handler pairs a nonce with a capability or ownership check (there is no unauthenticated AJAX surface), admin and data routes are `manage_options`-gated, outbound requests go only to fixed hosts, and token-gating is decided server-side from CardanoPress's asset cache — never from client-supplied wallet or policy data.

The assessment returned **0 Critical, 0 High, 1 Medium, 5 Low, 3 Info**. The Medium and the material Low-severity items were **resolved and deployed to production in this milestone**; the remainder are dispositioned below.

ID	Sev	Finding	Disposition
FIND-01	Medium	Public survey/intake endpoints trusted client-supplied forwarded headers (<code>X-Forwarded-For</code>) ahead of <code>REMOTE_ADDR</code> , allowing an attacker to spoof the IP, bypass rate limits, and flood the intake form (each accepted submit fires <code>wp_mail</code> + inserts a row)	Resolved. Client IP now derives from <code>REMOTE_ADDR</code> ; forwarded headers are honoured only behind an explicit trusted-proxy opt-in (<code>VALT_TRUSTED_PROXY</code> , right-most hop). Added IP-independent site-wide hourly caps on both endpoints, which also bound outbound email. Deployed + verified on prod.
FIND-02	Low	Mint recipient wallet address exposed via public REST post meta (<code>show_in_rest</code>) on the <code>song</code> CPT — harvestable by anonymous callers	Resolved. <code>valt_nft_wallet_address</code> set to <code>show_in_rest => false</code> (needed server-side only).
FIND-03	Low	<code>register_post_meta</code> auth callback checked the generic <code>edit_posts</code> capability, not the specific object	Resolved. Callback is now object-scoped: <code>current_user_can('edit_post', \$object_id)</code> .

ID	Sev	Finding	Disposition
FIND-04	Low	Docblocks advertised a "nonce" control on the intentionally-anonymous public endpoints, which was not (and cannot meaningfully be) enforced — a misleading-documentation risk	Resolved. Docblocks corrected to state the actual controls (honeypot + consent + rate limit).
FIND-05	Low	IDOR: a logged-in artist could reference another user's attachment/album by ID on their own song; at mint the media is read and pushed to IPFS	Resolved. Cover-art, audio, and album references are now ownership-checked (<code>post_author === current_user + correct post type</code>).
FIND-06	Low	<code>valt_deduct_points()</code> lacked a positive-amount guard (a negative amount would pass the balance check and credit the user) — not reachable via current callers	Resolved. Guard added at the function entry; the invariant no longer depends on callers.
FIND-07	Info	NFT lifecycle status enumerable by song ID	Accepted. These are ultimately public on-chain artifacts; negligible impact. No change.
FIND-08	Info	The token-gating trust anchor is the external CardanoPress dependency; a stale code comment referenced an interim guard mu-plugin no longer present	Operational. CardanoPress pinned $\geq$ v1.36.1 (the version that fixed the disclosed 0-day, see §4); the stale comment was corrected to document the real, upstream binding.

The assessment ran read-only against the source (no live traffic or exploitation against production). Its working notes are retained privately by Awen; PII and any reproduction detail are excluded from this public report, consistent with the milestone's redaction practice.

4. On-Chain / Wallet-Layer Review

Valt delegates all wallet code to CardanoPress: it verifies a CIP-8/COSE_Sign1 wallet signature, queries the chain via Blockfrost, and caches the result in per-user meta that Valt's token-gate reads. We traced that path end to end.

4.1 What is sound

- **Signature verification is genuine.** The wallet-connect flow performs full COSE_Sign1 validation — reconstructing the address from the public-key hash and doing a real ed25519 check. Proof of control of the signing wallet is cryptographic, not claimed.
- **Token-gating is server-side.** The decision reads a server-held asset set, never a client-supplied value, and non-holders are never sent the gated markup — content is withheld, not merely hidden with CSS (verified in `gating.php` and the gated shortcodes).
- **TLS verification is on** for every outbound call (NMRK, Blockfrost, Pinata, Stripe); no call disables certificate checking.

4.2 Two critical weaknesses in the wallet-authentication layer — now fixed upstream

The review identified two critical issues in the **third-party CardanoPress** wallet-connection handler, present through v1.36.0. In combination they permitted, with no key compromise:

- **Unauthenticated account takeover** — the login verified a CIP-8 signature over the connecting *wallet* address, but derived the WordPress account identity from a **separate stake-address value the signature never covered**. Since every Cardano stake address is public on-chain data, an attacker who signed a challenge with their own wallet could be logged in as any user simply by submitting that user's stake address — no secret required.
- **Authorisation / token-gating bypass** — that same unverified stake address drove the on-chain asset lookup the gate reads, so an attacker could have another account's holdings attributed to themselves and unlock gated content they do not own.

Resolved — responsibly disclosed and fixed upstream. Reported privately to the CardanoPress maintainer on **16 July 2026** with a full write-up, a working proof-of-concept, and a suggested patch, under a coordinated-disclosure window. The maintainer released **CardanoPress 1.36.1** on **17 July 2026** — a security release that derives the stake address server-side from the signature-covered wallet address and ignores the client-supplied value entirely,

closing both attack paths at the source. The release **credits Valt / Awen publicly**. Technical detail is included here now that the fix is public; it was withheld while the flaw was unpatched.

4.3 The response, end to end

Step	Detail
Found & reproduced	Traced end to end and reproduced against CardanoPress's own signature verifier — a forged request accepted, the victim's identity derived — proving the flaw rather than asserting it.
Responsibly disclosed	Reported privately to the maintainer (16 Jul 2026) with root cause, PoC and a suggested fix. No public detail while the vulnerability was unpatched.
Interim mitigation	While awaiting the fix, a first-party mu-plugin bound each login to the cryptographically verified wallet — oracle-verified 16/16, legitimate users unaffected — so Valt was protected throughout the disclosure window.
Resolved & hardened	Maintainer shipped 1.36.1 (17 Jul 2026) with the correct server-side fix. Valt updated local and production to 1.36.1 and removed the now-redundant interim mitigation. The fix protects every CardanoPress site , not just Valt.

4.3.1 Public evidence — independently verifiable

Don't take the captures below on faith; the primary record is the public release itself:

- **Release (durable record):** github.com/CardanoPress/cardanopress/releases/tag/v1.36.1 — commit **dcc1d94**. The notes describe the exact vulnerability and the fix, and the **Credit** line names the reporter.
- **Maintainer's announcement:** the CardanoPress maintainer's public post crediting the disclosure.

CardanoPress / cardanopress

CodeIssuesPull requestsActionsProjectsSecurity and qualityInsights

Releases / v1.36.1

1.36.1 — Security fix: wallet-login account takeover

Compare

pbwebdev released this 20 hours ago

v1.36.1

→ dcc1d94

Security release — please update immediately

CardanoPress 1.36.1 fixes a critical authentication vulnerability in the Cardano wallet login flow that could allow a pre-authentication account takeover.

Impact

The wallet login correctly verified a CIP-8 / COSE_Sign1 signature over the connecting wallet's address — but the WordPress account identity was taken from a separate stake-address value that the signature never covered. Because every Cardano stake address is public, on-chain data, an attacker who signs a login challenge with their own wallet could be logged in as any user simply by submitting that user's stake address — without knowing any secret.

Any site that uses CardanoPress wallet login for account creation or authentication is affected.

Severity: Critical (unauthenticated account takeover).

The fix

- The stake address used as the account identity is now derived server-side from the signature-covered wallet address, so it can no longer be substituted with someone else's.
- The previously-trusted stake_address request parameter is ignored entirely.
- Enterprise addresses (which carry no stake credential) are now rejected at login.

No configuration changes are required. Existing accounts continue to work unchanged — stake addresses are stable, so legitimate identities are unaffected.

Action required

1. Update to 1.36.1 as soon as possible.

2. Audit for prior compromise. This release ships a read-only audit script that flags any account whose stored stake address does not match the wallet it was created with. It is in the source repository (not bundled in the wordpress.org download):
[tools/audit-stake-binding.php](#)

Download it, place it on your server, and run with WP-CLI from your WordPress root:

```
wp eval-file /path/to/audit-stake-binding.php
```

Investigate any account reported as SUSPECT.

Credit

Thanks to Cullah (@CullahMusic, valt.digital) for responsibly reporting this vulnerability.

Full changelog: see `readme.txt`.

Assets

Source code (zip)

yesterday

Source code (tar.gz)

yesterday

© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

CardanoPress v1.36.1 release notes. Credit line: "Thanks to Cullah (@CullahMusic, valt.digital) for responsibly reporting this vulnerability." The release also ships a read-only `tools/audit-stake-binding.php` compromise-audit script. Source: github.com/CardanoPress/cardanopress/releases/tag/v1.36.1

6 / 10



4.4 Token-gating cache freshness — RESOLVED

The gate reads a cached asset set that the wallet layer rebuilt only on login or explicit sync — with no expiry. Left alone, access could outlive ownership: a holder who sells or transfers their NFT kept access until their next login, and a secondary-market buyer gained it late. This matters precisely because holdings move on other marketplaces. Valt now applies a **15-minute freshness window** (`gating.php`): when the cache is older, a **non-blocking** background refresh is queued through CardanoPress's own sanctioned refresh path, so gating tracks real on-chain ownership in both directions without adding latency to the page load. The misleading in-code comment that claimed the cache "cannot be spoofed" has been corrected to describe the actual trust model.

5. First-Party Controls Review

Every boundary in Valt's own code was reviewed by hand. Counts are for the shipped first-party code.

Control	Plugin	Theme	Assessment
AJAX nonce verification	13	—	13 of 13 handlers. No gaps.
Unauthenticated AJAX (wp_ajax_nopriv_*)	0	0	None in first-party code.
REST permission_callback	12 / 12	—	100% explicit.
Capability checks	8	3	Admin actions gated on manage_options .
Input sanitisation	42	1	Applied at every write path.
Output escaping	120	99	Two omissions found and fixed — §6.2.
Prepared SQL	~20	0 direct SQL	No interpolation of user input.
Ownership checks (beyond capability)		yes	Artist actions verify object ownership, defending against horizontal escalation.

Credential handling. Stripe secrets are wp-config constants only, never in the database, and masked in the settings UI — the reference pattern. NMKR and Pinata keys prefer a WP option (see §6.4). No secret appears in source or git history. The Stripe webhook is correctly public and verified by HMAC with a constant-time compare and a replay window.

6. First-Party Findings

6.1 Medium CSRF on destructive admin seeders — RESOLVED

Both data seeders performed state-changing writes on a plain GET ?confirm=yes with a capability check but no nonce; the page seeder overwrites site pages and removes Elementor data. Fixed with wp_nonce_url() + check_admin_referer() , matching the NFT Monitor's existing pattern. A regression test fails any admin page that acts on ?confirm= without a nonce.

6.2 Low Stored XSS in the artist dashboard — RESOLVED

\$song_album and \$song_duration were echoed unescaped while the adjacent line escaped correctly. Both derive from post meta, settable via Pods/wp-admin, bypassing the sanitising AJAX path. Fixed with esc_html() ; the fallback was changed from the HTML entity — (the reason escaping had been dropped) to a literal em-dash.

6.3 High NFT minted for an unsettled card payment — RESOLVED

Issue. The Stripe checkout handler (the secondary "pay with card" path — the primary Collect flow is NMKR Pay) acted on checkout.session.completed without checking that the payment had actually settled. Stripe fires that event for unsettled sessions too (delayed-notification methods complete as unpaid and settle later). An NFT could therefore be minted and delivered, and points awarded, for a payment that never landed.

Resolution. The handler now fulfils only when payment_status is paid (or no_payment_required for a legitimately free checkout); anything else is logged and skipped. This gates only the card-purchase path. On-chain ownership — secondary-market purchases, transfers, gifts — is unaffected and remains the sole basis for access, by design.

6.4 Low Minting credentials readable from the database — ACCEPTED

The NMKR API key and Pinata JWT prefer a WP option over the wp-config constant, so they live in wp_options and appear in the settings DOM. Any database disclosure or admin-panel XSS would expose minting credentials. Accepted for this milestone, gated before mainnet: the exposure requires an existing compromise, and the affected keys are pre-

production credentials with no mainnet minting authority. Migrating both to constant-only storage (mirroring Stripe) is a mainnet prerequisite.

6.5 Medium Unbounded pagination on public REST routes — ACCEPTED

`per_page / limit` on the public discovery, trending and leaderboard routes are cast to integer but never clamped. `?per_page=999999` forces a full-table query; unauthenticated and unthrottled, this is a denial-of-service amplifier (the data returned is already public). **Accepted for the testnet phase** with hosting-level rate limiting in front; clamping and a route argument schema are a mainnet prerequisite.

6.6 Low Payment idempotency is transient-based — ACCEPTED

The Stripe dedup key is a transient set after processing (a check-then-act race under webhook retries) with a shorter lifetime than the replay window. Concurrent retries could double-process. **Accepted for this milestone:** low likelihood on the testnet volume; a durable idempotency key and handling of the delayed-settlement `async_payment_succeeded` event are scheduled with the mainnet payment hardening.

6.7 Informational Rate limiting & perimeter tooling — PARTIALLY RESOLVED

The public **write** endpoints (feedback survey and artist intake) now carry application-level throttling — per-session/email/IP limits **plus IP-independent site-wide hourly caps** — added when the autonomous-style assessment showed the IP-based limits were bypassable via header spoofing (\$3.1, FIND-01), now deployed to production. Not yet throttled: the public **read** and point-earning routes; and no WAF / login-throttler / vulnerability monitor is installed. Those remain appropriate to defer for a pre-production deployment with no funds at risk, and are mainnet prerequisites.

7. Measures Implemented

The Output requires implementing measures, not only assessing. In this milestone:

- **Wallet-layer mitigation** deployed and oracle-verified (§4.3), closing two critical attack paths, with coordinated disclosure to the upstream maintainer.
- **Four first-party fixes shipped** — CSRF, stored XSS, unsettled-payment mint, and gating cache staleness.
- **Autonomous-style assessment run and its findings remediated** (§3.1) — the Medium rate-limit bypass and the material Low-severity items (public wallet-address exposure, object-scoped meta auth, attachment/album IDOR, points negative-guard, misleading nonce docs) were fixed and **deployed to production** this milestone.
- **Security invariants made executable.** `tests/SecurityTest.php` asserts every REST route has a permission callback, every AJAX handler checks a nonce, no `wp_ajax_nopriv` exists in first-party code, no destructive admin page acts without a nonce, no dangerous construct is used, and no credential pattern appears in source. A regression fails the build.
- **Continuous scanning.** semgrep runs on every push and pull request, SARIF retained as an artifact.
- **Secrets guardrail.** A `.gitignore` now blocks committing `wp-config.php`, `.env` or key material — the repository previously had no such protection.
- **Wallet component updated** to the latest release, with the mitigation re-verified against it.

8. Residual Risk & Mainnet Prerequisites

Milestone 2 committed to mainnet launch "following security review and QA". This report is that review. The following must be closed before mainnet, where real funds and custody change the risk calculus:

#	Prerequisite	From
1	✓ DONE. Upstream fix released (CardanoPress 1.36.1) and deployed to local + production; interim mitigation removed	§4
2	Move NMKR and Pinata credentials to constant-only storage; mask the settings field	§6.4
3	Clamp <code>per_page</code> / <code>limit</code> ; add REST argument schemas	§6.5
4	Durable payment idempotency; handle delayed-settlement (<code>async_payment_succeeded</code>)	§6.6
5	Partial. Write endpoints (survey/intake) now rate-limited (§3.1/§6.7); remaining: read/point-earning routes + perimeter tooling (WAF, login throttling, monitoring)	§6.7
6	Independent third-party review of the minting and gating paths before custody of mainnet assets	This is a first-party review; it is not a substitute for external audit at mainnet.

This assessment was performed by the project team, and now includes the wallet-connection layer that Valt's security model depends on. It is documented to the standard the Acceptance Criterion requires — *"manual security assessments documented by the team"* — and is explicitly not represented as an independent third-party audit.